+127 (and not -128 to +128, or -127 to +127, which are symmetrical). The implication is that if you try to negate a negative value, it does not always result in a positive value. For example, assume that the variable *i* is of type signed byte, and it holds the value -128; now, for the expression –*i*, the result overflows, and the end result is that the value of *i* remains the same (i.e., -128)!

To give you a familiar example related to this problem, consider the following straightforward implementation of the *abs* (absolute) function for integers:

```
int abs(int arg) {
            if(arg < 0)
                        return -arg;
            return arg;
}
```

It will not always return a positive value! This can cause crazy bugs as well. Let us look at an example from the documentation of the FindBugs tool. In Java, the *hashCode()* method for strings like *'polygenelubricants"* and *'DESIGNING WORKHOUSES"* will result in *Integer.MIN_VALUE*, so using the *abs()* method on the *hashCode()* method will give the same negative value! (FindBugs warns us of this problem with the rule 'RV: Bad attempt to compute absolute value of signed 32-bit hashcode'). Note that hash code is signed in Java. Hashcodes are naturally represented as unsigned numbers, because they are often calculated with bit manipulation, and processed as hexadecimal values. However, Java does not support unsigned integers, and hence hashcode uses *signed int* in Java. Note that this lack of support for unsigned integers in Java is a cause of numerous bugs; system programming requires extensive use of unsigned numbers and Java does get used for systems software development (e.g., for writing network protocol implementations, processing encoded data stored in native file formats, etc). Java programmers have to use various workarounds to circumvent this lack of support for unsigned integers in Java, and that causes bugs.

To understand the language peculiarities related to overflow, consider listing the operators that can overflow in languages like Java. In unary operators, the unary minus (-) operator can overflow for a minimum value, as we just discussed. How about unary plus (+)? It's a dummy operator (i.e., one that has no effect), so it does not overflow. Increment/decrement operators can overflow. If you consider the casts, downcasts to smaller types can overflow—for example, downcast from *int* to *short*. In binary operators, obviously, binary addition (+), subtraction (-) and multiplication (*) can overflow. How about division (/) and modulus (%) operators? Yes, division can overflow when the dividend is the minimum value of *integer*, and the divisor value is -1. Now for the surprise: the modulus operator (%) does not overflow even in case the dividend value is the minimum value of integer, and the divisor value is -1! This is because, in Java, the identity $a == (a / b) * b + (a \% b)$ holds true. So, $(a \% b) = a – ((a / b) * b)$ is true. Given that a is the minimum value of integer, and b is -1, the result of $(a \% b)$ is 0.

Let's evaluate the RHS of this equation for these values of *a* and *b:* the result for the expression (a / b) is the same value as *a;* so $(a * b)$ is $a * -1$, which again overflows, and the resultant value is *a;* finally, $a – a$ is 0. After seeing this explanation, you can understand why $a \% b$ does not overflow, but a / b does overflow. Yet, this behaviour is still unintuitive, isn't it?

Now let's look at the language peculiarities in handling data types that can cause bugs. Consider the following method, which is an abstraction of an actual code (and the resultant bug) found in a real-world enterprise project:

```
int foo() {
     return (false)? 1: null;
}
```

This code would result in a *NullPointerException* in Java! Intuitively, this code should never compile in the first place; a conversion from *null* to *int* does not make sense (e.g., the same code segment will result in a compiler error in C#). However, the code compiles in Java because of the unusual boxing and unboxing semantics in the language. This example is just to show that many aspects of programming are unintuitive, and they can result in bugs in the software. But why would anyone write such unusual code in an enterprise project? I don't know, but my experience with enterprise software shows that all kinds of such crazy bugs are present in real-world software that we pay so much money to buy. Whatever it is, Murphy's Law holds true for software too: "If anything can go wrong, it will"! END

By: S G Ganesh

The author works for Siemens (Corporate Research and Technologies), Bengaluru. You can reach him at sgganesh at *gmail dot com*.